

CHRONO SUPPORT FOR HUMAN-IN-THE-LOOP SIMULATION

Kyle Sha
Simulation-Based Engineering Lab

What counts as human-in-the-loop

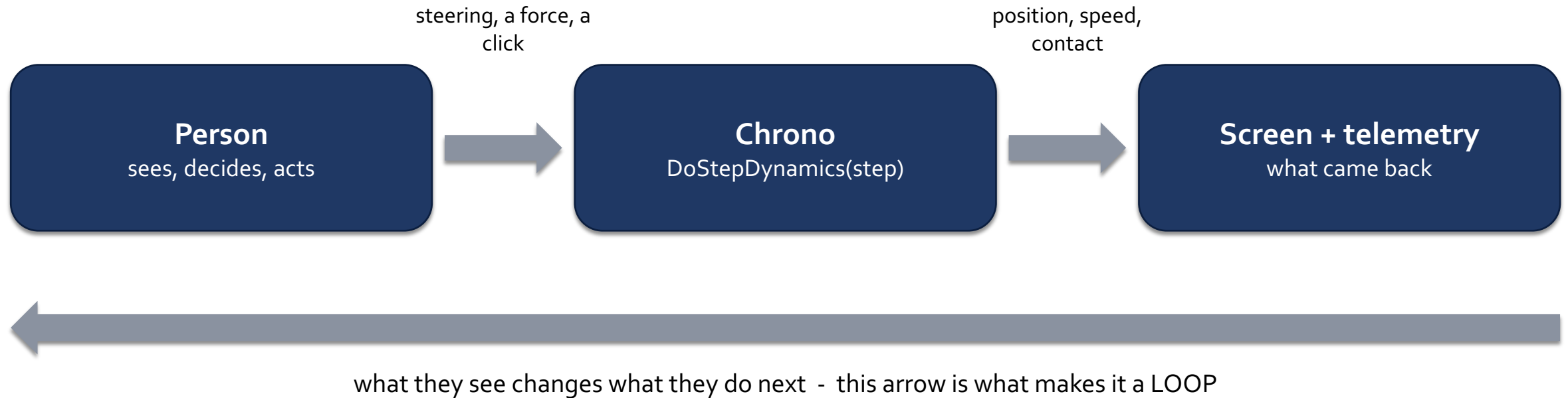
the definition this tutorial works to, and the three things it demands of a simulator

The definition this tutorial works to

- A simulation is human-in-the-loop when all three of these hold:
 - 1. a person's action changes the state of a RUNNING simulation,
 - 2. the simulation's response changes what that person does next, and
 - 3. the cycle closes fast enough that they can keep reacting.
- Each clause is load-bearing:
 - drop 1 and you have a visualisation - you are watching, not acting
 - drop 2 and the person is driving blind - open loop, not a loop
 - drop 3 and you have a batch job with a joystick attached

Nothing in that definition mentions vehicles. That is the whole point of the tutorial: a car, a crane, a gearbox and a quadruped's leg all satisfy it, and they satisfy it the same way.

The loop, and the clock around it



and every lap has to fit inside one wall-clock step

Break the first arrow and it is a movie. Break the return arrow and it is a recording session. Break the clock and the person cannot stay in the loop at all, which is why the clock comes first in this tutorial.

Not everything interactive is in the loop

- IN the loop - a person's input is a term in the dynamics, every step
 - driving the HMMWV; dragging a robot's limb while its controller resists
- ON the loop - a person configures and triggers, then watches it play out
 - the push rig: you choose where and how hard, the impulse itself is scripted
- Neither - interactive, but nothing is reacting
 - the top-down placement tool: kinematic, no dynamics, a scene-authoring tool

Worth saying out loud, because 'interactive' and 'in the loop' get used interchangeably and only one of them has a clock in it. All three are useful. Only the first two are what the phrase means.

Requirement 3 comes first: the clock

it decides whether the other two requirements are worth anything

What real time means in Chrono

- Chrono integrates as fast as it can. It has no notion of wall-clock time.
- $RTF = \text{wall time} / \text{simulated time}$
 - $RTF < 1$ - faster than real time. You have slack, and can sleep.
 - $RTF = 1$ - exactly real time.
 - $RTF > 1$ - slower than real time. No amount of sleeping fixes this.
- So holding the clock is two separate problems:
 - do not run FAST - sleep off the slack (easy, one call)
 - do not run SLOW - make the step cheap enough to have slack (the real work)

Unpaced, a dragged box in the manipulation demo travelled 88.7 m in one wall-clock second instead of 1.8 m. The physics was right. The experience was useless.

Measure it, then hold it (Lines 780 to 791, 684 to 698, 812 to 816)



```
# Console report: how far is the sim from wall time?
if step_number % report_steps == 0:
    wall = time.perf_counter() - wall_start
    # A ChVehicle measures its own real-time factor per step. The rover
    # and the crane do not, so it is measured here over the reporting
    # interval instead: same quantity, coarser sampling.
    if vehicle:
        rtf = vehicle.GetRTF()
    else:
        d_wall = wall - last_wall
        d_sim = sim_time - last_sim
        rtf = (d_wall / d_sim) if d_sim > 0 else 0.0

# Real-time setup (PART 1)
# -----
realtime_mode = REALTIME
if REALTIME == "vehicle":
    if vehicle:
        vehicle.EnableRealtime(True)
    else:
        # There is no vehicle to do the spinning, so fall back to the timer
        # that does the same thing from outside. Same soft real time, same
        # per-step measurement, no drift recovery either way.
        print('[realtime] REALTIME = "vehicle" needs a Chrono::Vehicle; '
              'using "per_step", which is the same policy.')
        realtime_mode = "per_step"
rt_timer = chrono.ChRealtimeStepTimer() # used when REALTIME == "per_step"
cum_timer = CumulativeRealtimeTimer()  # used when REALTIME == "cumulative"

# PART 1: spin in place for real time to catch up
if realtime_mode == "per_step":
    rt_timer.Spin(step_size)
elif realtime_mode == "cumulative":
    cum_timer.spin(system.GetChTime())
```

`ChRealtimeStepTimer.Spin(step)` sleeps off whatever is left of the step. `vehicle.EnableRealtime(True)` does the same thing from inside `Advance()`.

Per-step timers never recover time lost on a slow step, so a cumulative timer is offered too: it paces against total elapsed time and can catch up.

Show Time

- `REALTIME = "none"`: watch the drift column run away
- `REALTIME = "vehicle"`: drift pinned near zero, RTF at 1
- Same physics in both runs. The only difference is whether the process sleeps.

The drift column is the honest one. RTF can look fine on average while the sim has already lost a second of wall clock.

You can only sleep if you have slack

- Everything from here is about the second problem: making a step cheap.
- The two that actually moved the number on these demos:
 - contact - how much geometry the collision system is asked to resolve
 - the solver - how many iterations each step is allowed
- They pull against each other. Iterations buy stability and cost time; cheaper geometry buys time and can cost fidelity.
- The budget is fixed: one step of wall clock, minus what you want to leave for rendering.

Both numbers on the next two slides came out of the same robot, a Unitree Go2 loaded from URDF, on a laptop.

Where the time goes: contact

- Triangle meshes straight from the URDF: 427 contacts standing still, RTF 1.34 - slower than real time, before any human touches it
- The robot was resolving its own thighs against its own shins, every step.
- Convex hulls, plus the base link in its own collision family: 6 contacts, RTF 0.09
- Same robot, same controller, same step size. Roughly 15x the slack.

A URDF collision mesh is authored for a planner that asks 'do these overlap'. A real-time dynamics loop asks it every step, for every pair. Those are different jobs and the file does not know which one you are doing.

Where the time goes: the solver

- PSOR, 50 iterations: the robot slid across the floor as if friction were switched off.
- It was not a friction bug. Friction was under-resolved: the solver ran out of iterations before the tangential constraints converged.
- BARZILAIBORWEIN, 200 iterations: creep fell to 0.3 mm/s.
- Iterations are the price of stability, and you pay for them out of the same wall-clock step you wanted to sleep in.

This one is worth knowing because it does not look like a performance problem. It looks like wrong physics, and it sends you hunting through friction coefficients.

Requirement 1: a path into the state

how a person's action actually reaches the equations

Three ways a person's input enters a Chrono simulation

As driver inputs

ChDriver / ChInteractiveDriver
steering, throttle, braking

the vehicle demos (Parts 2-4)

IN the loop

As a force or constraint

AccumulateForce, ChLinkTSDA
on any body, any scene

the drag and push demos (Parts 9-10)

IN the loop

As a pose you set

SetPos / SetRot
kinematic placement

the placement tool

not in the loop

The middle column is the general one. A vehicle has a driver class because somebody wrote one; every other plant in this tutorial takes its human input as a force, which is why the same pattern reaches a crane, a gearbox and a quadruped.

The right-hand column is honest about itself: nothing reacts to a pose you set, so nothing closes the loop.

The keyboard, and what a keypress means (Lines 558 to 585)

```
elif INPUT_SOURCE == "keyboard":
    ### PART 2: keyboard through the Irrlicht window ###
    # W/A/S/D drive, 'C' centers steering, 'R' releases the pedals, 'L' locks
    # the current inputs. The ARROW keys are not driving controls: they belong
    # to the chase camera (zoom and orbit), which is a separate Irrlicht event
    # receiver. Easy to get wrong, because every other part of this tutorial
    # drives with the arrow keys - but those are OUR pygame console, not Chrono.
    driver = veh.ChInteractiveDriver(vehicle)
    driver.SetGains(4.0, 4.0, 4.0) # first-order lag from key target to applied input

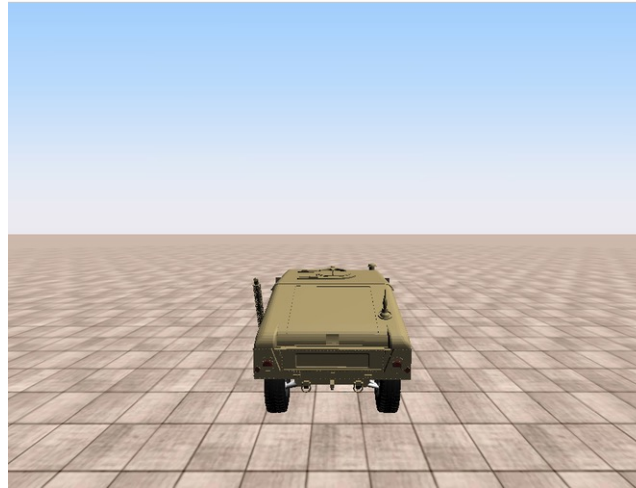
    # KEYBOARD_MODE picks what a keypress MEANS. This is the same question
    # PART 4 answers for its own device, so it is worth seeing both answers:
    #
    # "cumulative" each keypress nudges a target by a fixed delta, and the
    #               target stays where you left it. Historical Chrono
    #               behaviour, and what the deltas below configure.
    # "held"       the input follows the keys currently held down and ramps
    #               back when you let go, as in a driving game. This is the
    #               same "levels, re-sent continuously" model that
    #               operator_console.py uses over UDP in PART 4, and SetGains
    #               above is the ramp rate.
    #
    # "held" needs the KeyboardMode API (PyChrono build 1187 / Aug 2026 and
    # later). On an older PyChrono, say so and carry on cumulatively rather
    # than dying with an AttributeError.
    if KEYBOARD_MODE == "held":
        if hasattr(veh.ChInteractiveDriver, "KeyboardMode_HELD"):
            driver.SetKeyboardMode(veh.ChInteractiveDriver.KeyboardMode_HELD)
```

CUMULATIVE (the old default): a keypress NUDGES the input, and it stays where you left it. Fine for a test script, strange under a hand.

HELD: the input follows the keys currently down, like a driving game. That mode was added to Chrono for this tutorial and ships in PyChrono 10.0.0 build 1187.

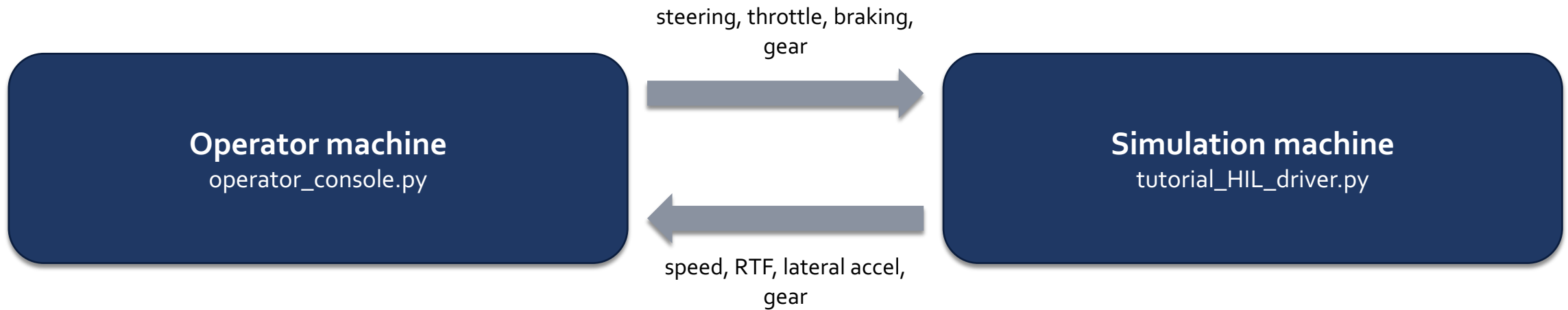
Show Time

- `INPUT_SOURCE = "keyboard", KEYBOARD_MODE = "held"`
- W/A/S/D drive. The arrow keys are the chase camera, not the car.
- Try "cumulative" and feel the difference a key-state API makes.



This is the whole definition satisfied in one window: input reaches the state, the state comes back on screen, and the timer holds the pace.

A device Chrono does not know about



Plain UDP datagrams, two processes, optionally two machines. Chrono never learns what is on the other end: a console, a phone, a motion platform, a wheel rig, another simulator.

The return arrow is the part people skip, and it is the one that makes the operator's device part of the loop instead of a remote control.

Reading a device Chrono does not know about (Lines 242 to 264)



```
class UdpInput:
    """Receive 'steering,throttle,braking[,gear]' text datagrams from an operator.
```

```

    Run operator_console.py in another terminal (or on another machine) and
    drive with the arrow keys there. The socket is non-blocking: the physics
    loop never waits for the operator. If no packet arrived since the last
    step, the previous target is held (zero-order hold).
```

```

    PART 6 added the optional fourth field. It is a single character naming a
    gear command (see hil_gearbox.Gearbox.COMMANDS) or '-' for "no command this
    packet", and unlike the three numbers it is an EVENT rather than a level:
    the operator presses '[' once and means one upshift, not "keep upshifting".
    take_gear_commands() below is what enforces that, by handing each command
    out exactly once.
```

```

    Keeping the field optional is what stops the change from breaking an older
    console: a three-field packet still parses.
    """
```

```

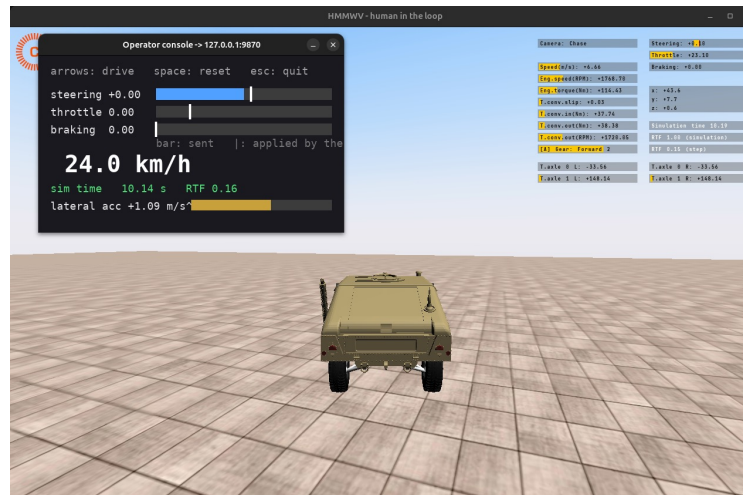
    def __init__(self, port=9870):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(("0.0.0.0", port))
        self.sock.setblocking(False)
```

Non-blocking is the whole trick: the simulation must never wait on a human. A late datagram means 'hold the last input', not 'stall the loop'.

Sample ONCE per step and hold it, or one physics step sees a different input than the next one for no reason the operator can perceive.

Show Time

- Two terminals on one laptop over localhost, or two machines over a LAN
- The console shows telemetry coming back while you drive



If the console's numbers stop moving, the loop is open and the operator is guessing. That is the failure this arrow exists to prevent.

Reaching into the scene with a mouse

what Chrono provides today, the one line it is missing, and what that buys

What Chrono gives you today

- `ChRealtimeStepTimer` - the pacing, exposed to Python
- `ChInteractiveDriver` - keyboard and joystick input, with `KeyboardMode::HELD`
- `ChCollisionSystem::RayHit` - click-to-pick, exposed to Python
- `AccumulateForce`, `ChLinkTSDA` - a human's pull as a term in the dynamics
- `ChVisualSystemIrrlicht::AddUserEventReceiver(IEventReceiver*)` - the socket for any input device, and it IS bound in Python
- What is NOT there: any built-in mouse pick-and-drag. No module in the tree implements one.

That last line is the gap this tutorial walks into. Everything needed to build one is already exposed; what is missing is the ability to receive the mouse event in the first place.

The gap, and the one line that closes it (native_pick.py lines 10 to 26)



```
# chrono_swig/interface/irrlicht/ChModuleIrrlicht.i
+%feature("director") irr::IEventReceiver;
#include "IEventReceiver.h"

# The module ALREADY compiles with directors enabled:
# %module(directors="1", threads="1") irrlicht
# and AddUserEventReceiver is ALREADY bound. Without the
# director, though, Python gets an abstract class:
# >>> ci.IEventReceiver()
# AttributeError: No constructor defined - class is abstract
# so there is a socket, and nothing you can plug into it.

class MouseReceiver(irr.IEventReceiver):
    """No Quartz, no CGWindowList, no NSEvent. No macOS. Just Irrlicht."""

    def __init__(self):
        super().__init__()
        self.x = self.y = 0
        self.down = False
        self.pressed = False          # edge, consumed by the loop
        self.keys = set()

    def OnEvent(self, ev):
        if ev.EventType == irr.EET_MOUSE_INPUT_EVENT:
            self.x, self.y = ev.MouseInput.X, ev.MouseInput.Y
            if ev.MouseInput.Event == irr.EMIE_LMOUSE_PRESSED_DOWN:
                self.down = self.pressed = True
            elif ev.MouseInput.Event == irr.EMIE_LMOUSE_LEFT_UP:
                self.down = False
```

Without it, reading the mouse means going around Chrono to the operating system: 135 lines, macOS only, and it breaks when another window takes focus.

With it, the same capability is 56 lines of ordinary PyChrono. Upstream PR to follow this talk.

Picking a body, then pulling on it (Lines 338 to 349, 488 to 502)

```
def pick_along_ray(system, start, end):
    """The primitive a mouse click would use. Returns (body, world_point) or None."""
    res = chrono.ChRayhitResult()
    system.GetCollisionSystem().RayHit(start, end, res)
    if not res.hit:
        return None
    body = chrono.CastToChBody(res.hitModel.GetContactable())
    if body is None:
        return None
    p, nrm = res.abs_hitPoint, res.abs_hitNormal
    return (body, chrono.ChVector3d(p.x, p.y, p.z),
            chrono.ChVector3d(nrm.x, nrm.y, nrm.z))

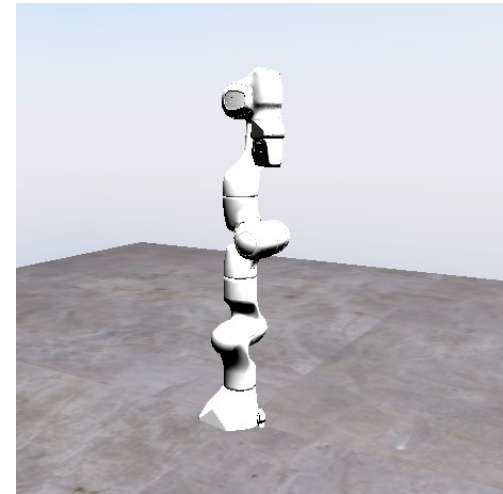
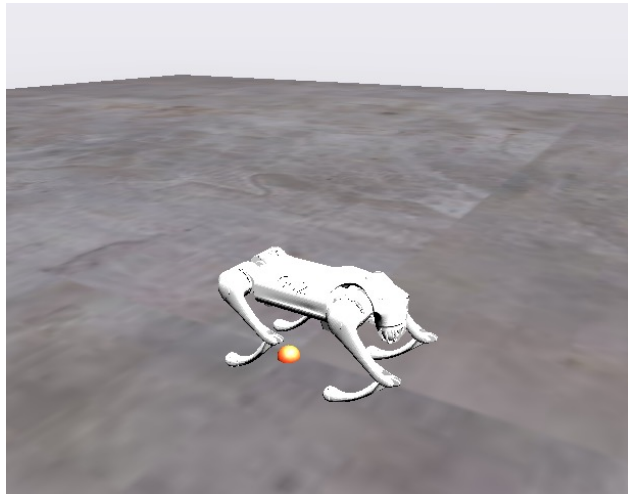
omega = getattr(self.system, "grab_omega", GRAB_OMEGA)
m = max(body.GetMass(), 1e-3)
k = m * omega * omega
c = 2.0 * m * omega * GRAB_ZETA
# Belt and braces against tunnelling: a grabbed body may not move faster
# than contact detection can keep up with at this step size.
self._had_limit = body.GetLimitSpeed() if hasattr(body, "GetLimitSpeed") else False
body.SetLimitSpeed(True)
body.SetMaxLinVel(GRAB_MAX_SPEED)
self.spring = chrono.ChLinkTSDA()
self.spring.Initialize(self.handle, body, False, point, point)
self.spring.SetRestLength(0.0)
self.spring.SetSpringCoefficient(k)
self.spring.SetDampingCoefficient(c)
self.system.AddLink(self.spring)
```

RayHit is the pick. It is already in PyChrono and it is exact: the same primitive a native mouse handler would call.

The spring is the part that has to be got right. Gains must scale with the mass being pulled: a stiffness tuned on a Go2 calf gave a Panda link 21,870 N/m, which is 24,300 m/s² and straight through the floor in one step.

Show Time

- Drag a leg while the stance controller fights back
- Drag a Franka link, hand-guided or fully unactuated
- Yanking a link into the floor: 35.14 m/s before the gains were bounded, 3.25 m/s after



This is the clearest case of the definition on the whole deck: you pull, the controller resists, you feel it resist, and you pull differently.

Taking the human out of the part that must repeat (Lines 264 to 283)



```
def fire(self, cfg, t):
    body = cfg.body
    self.active = (body, cfg.force(), chrono.ChVector3d(cfg.local_point),
                  t + cfg.duration)
    self.fired_at = t
    return self._slot(body)

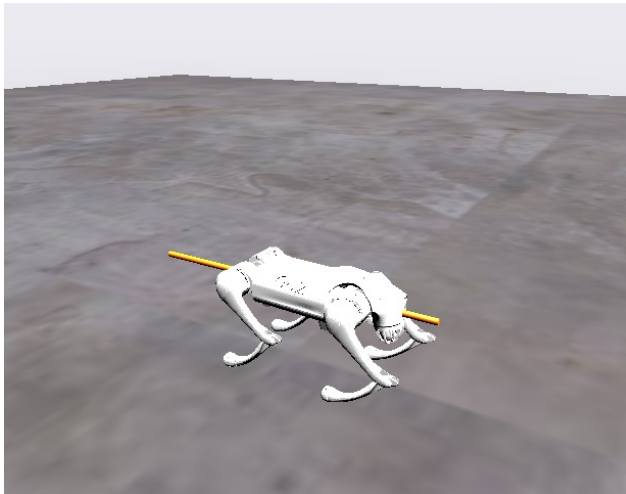
def update(self, t):
    """Call every step, BEFORE DoStepDynamics. Returns True while pushing."""
    if self.active is None:
        return False
    body, force, local, t_end = self.active
    idx = self._slot(body)
    body.EmptyAccumulator(idx)          # Chrono never clears these for you
    if t >= t_end:
        self.active = None
        return False
    # The point travels with the link: a shove on the shoulder stays on the
    # shoulder even as the shoulder is being shoved away.
    body.AccumulateForce(idx, force, body.TransformPointLocalToParent(local),
```

A freehand drag cannot answer 'how hard can I shove it', because no two drags are the same. The force depends on how fast your hand moved and how long you held the button.

So the person keeps WHERE and WHICH DIRECTION, and the impulse $J = F * dt$ is scripted and printed. That is a number you can put in a report and hand to somebody else.

Show Time

- Forward at the base COM: 325 N recovers, 330 N does not
- Sideways: 280 N recovers, 290 N does not
- Same impulse, applied 5 cm higher up the torso: the verdict flips



That last line is the reason a human is still in this one. Where to push is a judgement call, and it changes the answer as much as how hard you push does.

How far this goes

- PART 6 - change gear. A gear is not a fourth float: it belongs to the transmission, and the human sets it through a different API.
- PART 7 - drive somewhere real. SCENE = "mcity" puts the car in the Mcity digital twin.
- PART 8 - control something that is not a car. PLANT = "rover" or "crane", same three floats, no vehicle class involved.
- PARTS 9-10 - reach into the scene: drag a limb, push a robot, place a scene.

The through-line: the loop never changed. What changed was where the human's numbers were injected.

Where this goes next

- Chrono::HIL (github.com/zzhou292/chrono-HIL) - driving rigs, steering wheels, multi-user sessions, the same three floats
- The SWIG director change, as an upstream PR
- A mouse pick-and-drag utility, so that every PyChrono user does not write the same 56 lines
- Policy in the loop: a learned controller on one side, a person perturbing it on the other

The gap this talk found is small and specific, which is the good kind: one line to receive the event, and a utility on top of primitives Chrono already has.

Any questions?

University of Wisconsin - Madison

Thank you.

`github.com/ksha23/chrono-hil-tutorial`